



ugr

Universidad
de Granada

SEMINARIO S-codigos-qr

CÓDIGOS QR

Autor:

Mario Casas Pérez

1. Funcionamiento de los códigos QR	3
2. Programa para crear códigos QR	3
3. Bibliografía	7

1. Funcionamiento de los códigos QR

Los códigos QR (Quick Response) son códigos de barras bidimensionales que guardan datos tanto en el eje horizontal como en el vertical. Su estructura se compone de módulos (cuadrados negros y blancos) que puestos en una cuadrícula representan información codificada. Para obtener una lectura fiable (incluso cuando parte del código pueda estar dañado o sucio), los códigos QR tienen algoritmos de corrección de errores basados en el método Reed-Solomon. Esto permite que dependiendo del nivel de corrección de error seleccionado (L, M, Q o H) se puedan recuperar datos aunque parte del código esté comprometido. Además, el proceso de codificación consiste en transformar la información (texto alfanumérico, numérico o binario) en una secuencia de bits, que luego se organiza en la matriz del QR, integrando la redundancia necesaria para la corrección de errores.

2. Programa para crear códigos QR

Se ha realizado un programa en Python el cual usa la biblioteca `qrcode` para generar códigos QR a partir de una cadena de texto y guardarlo en un archivo de imagen. Los pasos a seguir para hacerlo han sido los siguientes:

- **Definición de la función `generaQR`:** este método recibe como parámetros la cadena de texto que se quiere codificar en el QR y el nombre de archivo de salida en el que se guardará la imagen del código.
- **Configuración y creación del código QR:** se configura un objeto de la clase `QRCode` siguiendo una serie de pasos.
 - **`version = 1`:** se define el tamaño mínimo del código QR. La versión 1 crea un código de 21x21 módulos, lo cual es suficiente para cadenas cortas (como nuestro nombre).
 - **`error_correction=qrcode.constants.ERROR_CORRECT_L`:** usamos un nivel de corrección bajo (L) permitiendo codificar más datos pero con menor tolerancia a errores o daños físicos en el código.
 - **`box_size = 10`:** se establece el tamaño en píxeles de cada módulo (cuadro) del código, teniendo este parámetro un impacto directo en el tamaño final de la imagen.
 - **`border = 4`:** se determina el ancho del borde en número de módulos. Este margen blanco alrededor del código es importante para que los lectores de QR lo identifiquen bien.
- **Adición de datos y generación de la imagen.**
 - **`qr.add_data(cadena)`:** añadimos la cadena de texto que se desea codificar. Se prepara el objeto `qr` para incluir la información dentro del patrón del código.

- **qr.make(fit=True):** con el parámetro *fit = True* indicamos que se calcule automáticamente el tamaño mínimo necesario para que todos los datos se metan correctamente. Esto nos garantiza que aunque se definiera una versión inicial, esta se ajuste si fuera necesario.
- **qr.make_image(fill_color="black", back_color="white"):** este método crea la imagen usando negro para el contenido y blanco para el fondo, siendo esto lo más común para la mayoría de los códigos QR. No obstante, tal y como hemos visto en clase, la biblioteca permite configurar colores personalizados, lo cual es útil para adaptarse a la identidad visual o requisitos concretos de diseño.
- **Guardado de la imagen y retorno:** con *imagen.save(nombre_archivo)* se escribe la imagen generada en el disco bajo el nombre descrito. Además, esta función devuelve el nombre del archivo.
- **Método principal:** se le indicará al usuario que inserte la cadena que quiere codificar (en este caso mi nombre) y el nombre del archivo de imagen que se generará. Luego, se llama a la función *generaQR* y finalmente se le envía un mensaje al usuario de que el código QR ha sido generado correctamente.

El código del programa es el siguiente.

```
import qrcode

def generaQR(cadena, nombre_archivo):
    qr = qrcode.QRCode(
        version=1,
        error_correction=qrcode.constants.ERROR_CORRECT_L,
        box_size=10,
        border=4,
    )

    qr.add_data(cadena)
    qr.make(fit=True)

    imagen = qr.make_image(fill_color="black", back_color="white")
    imagen.save(nombre_archivo)
    return nombre_archivo

if __name__ == "__main__":
    cadena = input("Introduce el texto a codificar en el QR: ")
    nombre_archivo = input("Introduce el nombre del archivo: ")

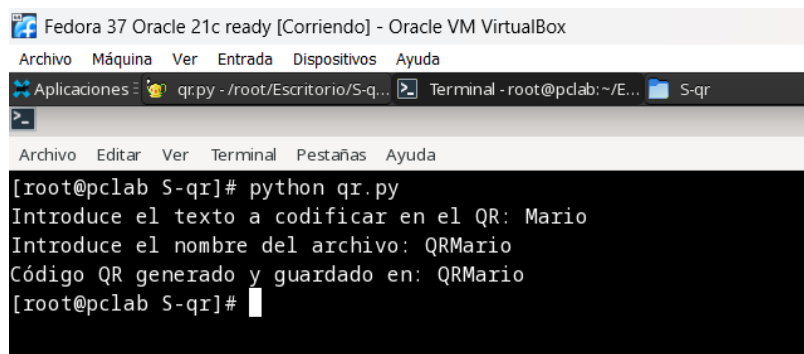
    generaQR(cadena, nombre_archivo)
    print(f"Código QR generado y guardado en: {nombre_archivo}")
```

Pasamos a ejecutar el programa. Para ello, lo primero de todo será garantizar de que *pip* esté instalado y que además tenemos las dependencias necesarias para ejecutar el programa (tener instalada la librería *qrcode*). Ejecutaremos lo siguiente (en mi caso, en *Fedora*).

```
sudo dnf install python3-pip

pip install qrcode[pil]
```

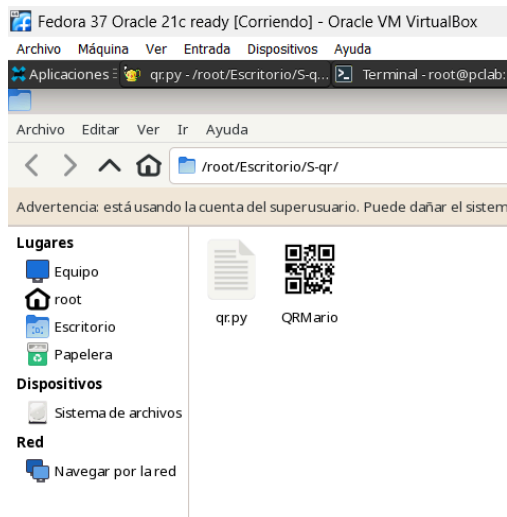
Luego, pasamos a la ejecución del programa. Nos vamos a donde tenemos nuestro código y ejecutamos lo siguiente.



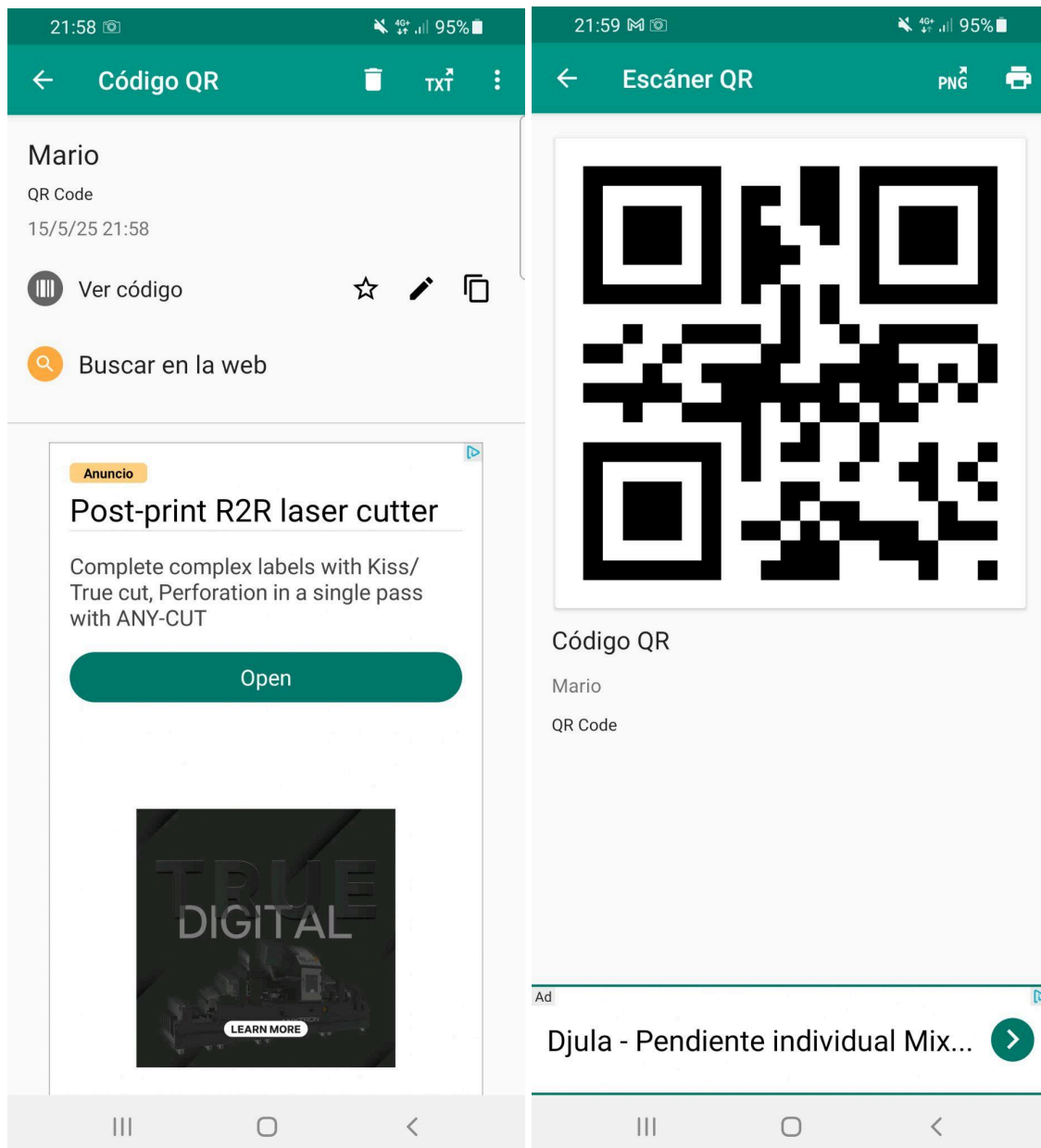
```
Fedora 37 Oracle 21c ready [Corriendo] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
Aplicaciones  qr.py - /root/Escritorio/S-q...  Terminal - root@pclab:~/E...  S-qr
Archivo  Editar  Ver  Terminal  Pestañas  Ayuda

[root@pclab S-qr]# python qr.py
Introduce el texto a codificar en el QR: Mario
Introduce el nombre del archivo: QRMario
Código QR generado y guardado en: QRMario
[root@pclab S-qr]#
```

Podremos ver como se ha creado un archivo *.png* donde se encuentra el código QR generado.



Ahora, con un programa que lee códigos QR pasamos a escanearlo (en mi caso usaré el programa Escáner QR disponible en Android). Una vez escaneado podemos ver como aparece nuestro nombre.



3. Bibliografía

https://swad.ugr.es/swad/tmp/8s/sA1f_Rd9doFJV_e9U0XF44AY9NzcJQq6ownpBAaEg/S-co-digos-qr.pdf

https://es.wikipedia.org/wiki/C%C3%B3digo_QR